

Homework 3 - Divide and Conquer

Course: CIS 387 - Analysis of Algorithms - Fall 2026

Instructor: Dr. Aaron Mininger

Submission Instructions

Due Date: October 8 at 11:59pm

Zip up the 3 java files and upload to Canvas. **Your name must be at the top of each java file**

Note: You may not use any AI assistance or direct outside help on this assignment.

For 2 extra credit points, answer the following questions (leave a comment on Canvas):

1. Give an estimate of how much time you spent on this assignment.
2. How difficult did you find this assignment?
3. Was anything unclear, or is there any other feedback you would give?

Your responses will be helpful feedback for refining these assignments in future courses.

Grading Information

Points	Description
20	Effort: Does the program compile and run? Was a reasonable attempt made?
10	Part 1: findFixedPoint returns the correct answer.
10	Part 1: findFixedPoint correctly uses recursive divide and conquer.
5	Part 1: There is a comment comparing the results of DoublingTest
10	Part 2: closestPair (1D) returns the correct answer.
10	Part 2: closestPair (1D) correctly uses recursive divide and conquer.
15	Part 3: closestPair (2D) returns the correct answer.
10	Part 3: closestPair (2D) correctly uses recursive divide and conquer.
5	Part 3: The solution includes the optimization described in list item 7
5	Part 3: The solution includes the optimization described in list item 8

Setup

All the code and files needed for this homework are available on the course git repository:

<https://github.com/amininger-mu/CIS387F26Code>

You will see starter files for each part that you should fill in. In addition, there are test files you can use to test each part (you should make additional test files for larger cases!).

1. Fixed Point

Consider an array of *distinct* integers in sorted order.

A **fixed point** is where an element's value equals its index $A[i] == i$.

For example, given the array `[-10, -5, -2, 0, 3, 5, 8, 11, 21]`, the fixed point is 5.

Directions:

You are given a file `FixedPoint.java` which has a naive solution (linear scan through the array). Implement a recursive divide and conquer solution to this problem starting with the function `int findFixedPoint(int[] arr)`.

Once you have completed your implementation, look at `DoublingTest.java` to see that it calls `FixedPoint::runTest(int n, boolean fast)`. This will do 1000 random fixed point trials and return the total time spent. Run both:

- `java DoublingTest.java -s` (slow version)
- `java DoublingTest.java` (fast version)

At the top of `FixedPoint.java`, leave a comment describing the following for each version (the given slow version, and your faster D&C version):

- What is the big-O complexity of that version?
- What ratio values did you see for that version (focus on the last few).
- Do the observed ratio values match your expectations, given the Big-O complexity? (explain)

Testing

The `FixedPoint.java` main function will read in the test case from standard in. It will first read in the solution, then a list of integers to create the array. It will then print the results.

You can feed in a test case using:

```
cat fp_test1.txt | java FixedPoint.java
```

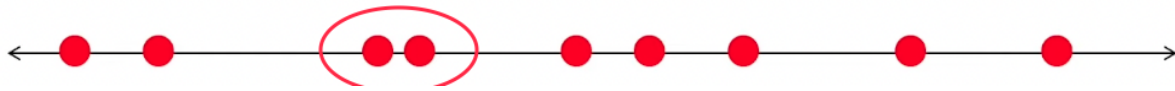
I have provided 1 test case file, but you should make others to test your algorithm. You can also run `java FixedPoint.java -s` to run the slow (naive) version of the code.

2. Closest Pair of Points (1D)

This is a warmup activity for part 3 (the 2D case).

Consider a set of n distinct x values on a number line. Find the closest pair of values (minimum difference).

Example:



Directions:

You are given a file `ClosestPair1D.java` which has some boilerplate code. You will implement the function `Pair closestPair(List<Double> values)`. The function must return a result `Pair` object that has the following structure:

```
// Result structure containing a pair of values and their separation distance
record Pair (double x1, double x2, double distance) { }
```

If you have not seen a Java record before, it acts like a struct or dataclass in other languages. It defines an immutable object with a list of attributes (here `x1`, `x2`, and `distance`), and implements many of the default Java methods automatically (constructor, `toString`, `equals`).

To create a `Pair`, do `new Pair(x1, x2)`, and it will compute the distance for you.

To begin, sort the list of points (using `Double::compare` as the comparator).

Then, implement a standard divide and conquer approach:

1. Add the appropriate base cases if there are 2 or fewer points in the range.
2. Find the midpoint and split the array in half.
3. Recurse on each half, finding the closest pair.
4. Combine the results, make sure to consider the case where the closest pair crosses the mid-point.

Testing

The `ClosestPair1D.java` main function will read in a list of doubles from standard in, then print out the closest pair of points using each version. It will check if the two versions produced the same result.

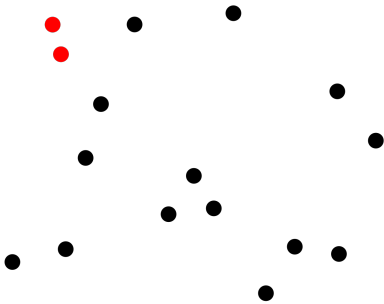
```
cat cp1_test1.txt | java ClosestPair1D.java
```

I have provided 1 test case file, but you should make others to test your algorithm.

3. Closest Pair of Points

Consider a set of n distinct points on the x,y plane. Find the closet pair of points (minimum distance apart).

Example:



Note that the combine step is significantly more complex than the 1D case, since multiple points in each half could be involved in the solution. (points close in the x dimension may be far apart in the y dimension). The general approach will be to find the closest pair from each half separately, then only test points that are close enough to the middle that they could possible beat the best recursive result.

Background:

You are given a file **ClosestPair2D.java** which has a naive solution (iterates over every pair of points). Review that implementation to get a sense of how the **Point** and **Pair** records work.

```
// an x,y point structure
record Point (double x, double y);

// function that calculates straight line distance between 2 points
double calcDistance(Point p1, Point p2);

// 2 Point comparators: on x coordinate, and y coordinate
//   to use, call list.sort(compareX);
int compareX(Point p1, Point p2);
int compareY(Point p1, Point p2);

// Result structure containing a pair of points and their distances
//   a provided constructor new Pair(p1, p2) will compute the distance for you
record Pair (Point p1, Point p2, double distance);
```

Directions:

Implement a recursive divide and conquer solution to this problem, starting with the function **Pair closestPair(List<Point> points)**. You should start by sorting the list of points by *x coordinate* (using **compareX**). The recursive part should be similar to your solution from part 2, except the combine step is more involved.

The overall strategy will be to only consider points close enough to the split boundary (midpoint *x coordinate*) that they could beat the best pair found so far. Detailed information is on the next page. If you want to challenge yourself, try to make progress without looking ahead.

Testing

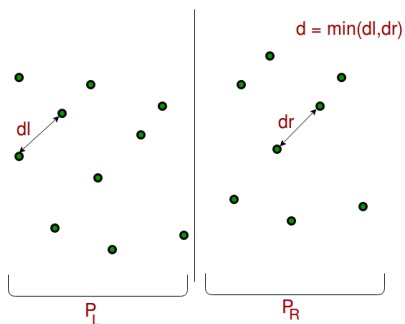
The **ClosestPoint2D.java** main function will read in a list of double pairs from standard in, then print out the closet pair of points. It will call both versions, then compare the results to see if they match.

```
cat cp2_test1.txt | java ClosestPoint2D.java
```

I have provided 1 test case file, but you should make others to test your algorithm.

Guide for Solving Closest Point in 2D

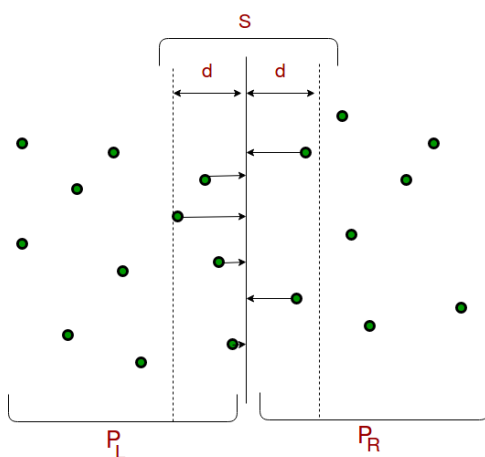
After doing the recursive calls, you will have the closest pair found in the left and right halves:



The only other case you need to consider is when the closest pair involves 1 point on the left side, and 1 point on the right side.

The key insight is this: Given the min distance d found so far, any point p where $(|\text{mid.x} - p.x| > d)$ cannot possibly be part of a closer pair that crosses the middle.

So, we only check points that are in the region shown below:



Required Steps:

1. Make sure that before you start, you sort the points according to the x value.
2. After checking the base cases, get the closest pair in the left and right halves. (d_l and d_r)
3. Determine the best pair between the two (smallest distance d)
4. Create a new, empty list of points called **strip**
5. Add all points that are within distance d of the mid line into the strip.
6. Iterate over all pairs in the strip to find the closest two
7. Further optimization: Sort the points in the strip along the y dimension, then you can stop looking at points that are greater than d distance apart in the y dimension.
8. Further optimization: Create two lists - a right strip and left strip. Sort them by y , then check all pairs with one point in left strip and one point in right strip (keeping the early exit condition from 7).